

Övningsprogram 12, Ping-97

Föreläsning 10

Ändrad
23 Sep
10:02

Hi-Lo igen! Denna gång ska datorn gissa.

```

1  Nu ska vi leka hi-lo!
2  Tänk på ett tal mellan 1 och 100!
3  Jag gissar.
4  Du svarar H (högt), L (lågt) eller R (rätt)!
5
6  Jag gissar på: 50
7  Högt? Lågt? Rätt?! : H
8  Oj då!
9  Jag gissar på: 25
10 Högt? Lågt? Rätt?! : L
11 Äsch!
12 Jag gissar på: 37
13 Högt? Lågt? Rätt?! : h
14 Oj då!
15 Jag gissar på: 31
16 Högt? Lågt? Rätt?! : k
17 Förstår inte "K"!
18 H = för högt, L = lågt och R = rätt.
19 Jag gissar på: 31
20 Högt? Lågt? Rätt?! : l
21 Äsch!
22 Jag gissar på: 34
23 Högt? Lågt? Rätt?! : r
24 Men...Jag vann!
24
25 Ha!
```

Det finns nästan inget nytt i detta exempel. Jag har stoppat in lite smått och gott av sådant vi redan gått igenom. En `switch`-sats, en liten funktion, en `do`-loop, etc.

Datorns strategi är att hålla reda på det högsta och minsta tänkbara värdet. Datorn lägger sin nästa gissning mitt emellan `high` och `low`.

```

1  /*
2  Övnings exempel 12, Ping-97.
3  Jan Tångring, Umeå 9/9 1997
4
5  Spela Hi-Lo. Datorn gissar.
6
7  Nyheter:
8  - toupper()
9  - blanktecken i scanf (matchar tomrum)
10 - strategi -- spara hi & lo, gissa mitten
11 */
12
13 #include <stdio.h>
14 #include <ctype.h>
15
16 #define MIN 1
17 #define MAX 100
18
19 int smartGuess(int low, int high)
20 {
21     return (low + high) / 2;
22 }
23
24 void main()
25 {
26     int high = MAX;
27     int low = MIN;
28     int guess;
29     char feedback;
30
31     printf("Nu ska vi leka hi-lo!\n");
32     printf("Tänk på tal mellan %d och %d!\n", MIN, MAX);
33     printf("Jag gissar.\n");
```

```

34     printf("Svara H (hög), L (låg) eller R (rätt)!\n\n");
35
36     do {
37         guess = smartGuess(low, high);
38
39         printf("Jag gissar på: %d\n", guess);
40         printf("Högt? Lågt? Rätt?! : ");
41
42         scanf(" %c", &feedback);
43         feedback = toupper(feedback);
44
45         switch (feedback) {
46             case 'H' :
47                 printf("Oj då!\n");
48                 high = guess - 1;
49                 break;
50             case 'L' :
51                 printf("Äsch!\n");
52                 low = guess + 1;
53                 break;
54             case 'R' :
55                 printf("Men...\n");
56                 break;
57             default :
58                 printf("Förstår inte \"%c\"!\n", feedback);
59                 printf("H = högt, L = lågt och R = rätt.\n");
60                 break;
61         }
62     } while (feedback != 'R');
63     printf("Jag vann!\n\nHa!\n");
64 }

```

Lite nytt finns det ändå: funktionen `toupper()` omvandlar från små bokstäver till stora. Jag omvandlar användarens inmatning (`feedback`) till stora bokstäver på rad 43. Därmed spelar det ingen roll om hon matar in stora eller små bokstäver.

Utan `toupper()` skulle inte programmet känna igen små bokstäver, eftersom `switch`-satsen använder stora bokstäver som `case`.

Funktionen `toupper()` finns i `ctype.h`. Mer om `ctype.h` strax!

Ät blanka, `scanf`!

Funktionen `scanf` har många trix på lager. Varje gång jag bläddrar i manualen hittar jag nåt nytt.

På rad 42 läser programmet in användarens bedömning som en `char` med `%c` till variabeln `feedback`. Denna teknik kan leda till problem, som vi bland annat sett i [Gruppövning 1e](#).

Problemet är att användaren trycker på *[Return]* när han matat in tecknet. Och den tangenten läser `%c` in som en `char` med ASCII-värdet 10!

I mitt lösningsförslag till [Gruppövning 1e](#) löste jag problemet genom att läsa in nyradstecknet till variabeln `dummy`.

Men det finns ett betydligt enklare sätt! Funktionen `scanf` har förutsett problemet, och bjuder på en lösning. Titta på rad 42 igen. Där finns ett blanktecken före `%c`, och detta lilla blanktecken löser problemet.

Blanktecknet säger åt `scanf` att äta upp allt tomrum som står i vägen för nästa ordentliga tecken. Som tomrum (eng *white space*) räknas blanksteg, nyrad och tabtecken.

ctype.h

Funktionen `toupper()` ligger i biblioteket `ctype.h`. Där finns också `tolower()` som jobbar åt andra hållet.

I `ctype.h` finns även ett gäng funktioner som avgör vilken kategori av tecken som en `char` tillhör. Funktionerna returnerar *sant/falskt* (1/0):

- `isalpha(c)` -- A--Z, a--z
- `isupper(c)` -- A--Z
- `islower(c)` -- a--z
- `isdigit(c)` -- 0--9
- `isxdigit(c)` -- 0--9, A--F, a--f
- `isspace(c)` -- "tomrum": ASCII 9--13,32
- `ispunct(c)` -- interpunktering
- `isalnum(c)` -- `isalpha(c)` eller `isdigit(c)`
- `isprint(c)` -- "skrivbara": ASCII 32--126
- `isgraph(c)` -- skrivbara, utom blanktecknet (ASCII

32)

- `iscntrl(c)` -- ASCII 0--31,127
- `isascii(c)` -- ASCII 0--127 (allihop)

! Ingen av funktionerna i `ctype.h` har den blekaste aning om vad ett åttabitarstecken är. ÅÄÖ är alltså varken "alpha" eller "upper". !

Feta siffror

Det finns en konvention om text på WWW: text som ramas in med paret `<b>` och `</b>` ska skrivas ut **fett**.

Följande program fetar upp siffror som ska ut på WWW. Programmet utnyttjar `isdigit()` för att hitta siffrorna.

```
1 #include <stdio.h>
2 #include <ctype.h>
3
4
5 void print_fat_if_digit(char ch)
6 {
7     if isdigit(ch)
8         printf("<b>%c</b>", ch);
9     else
10        printf("%c", ch);
11 }
12
13 void main()
14 {
15     int ch;
16
17     while ((ch = getchar()) != EOF)
18         print_fat_if_digit(ch);
19 }
```

En körning ser ut på följande sätt

```
1 Siffror - exempelvis 123 - ska fetas!
2 Siffror - exempelvis <b>1</b><b>2</b><b>3</b> - ska fetas!
3 ^d
```

Om Explorer eller Navigator skulle se den texten innan jag manipulerat den, skulle de visa följande:

```
Siffror - exempelvis 123 - ska fetas!
```

Villkorsuttrycket

Ibland är det praktiskt att ersätta en `if`-sats med ett villkorsuttryck. I stället för:

```
if (x > y)
    max = x;
else
    max = y;
```

kan man skriva:

```
max = x > y ? x : y;
```

Villkorsuttrycket består av tre delar:

Villkor ? *A* : *B*

Värdet av villkorsuttrycket är *A* eller *B* beroende på om villkoret är sant.

Vi kan använda villkorsuttrycket i programmet som fetar siffror. Vi kan byta ut rad 18 mot

```
18 printf(isdigit(ch) ? "<b>%c</b>" : "%c", ch);
```

Varning för plottrig programkod! Läs sakta. Följ texten med fingret. Leta reda på ? och :.

Beroende på om `isdigit(ch)` är sant, kommer där antingen att stå

```
printf("<b>%c</b>", ch);
```

eller

```
printf("%c", ch);
```

En text som är inramad med " kallas *sträng* (eng *string*). Vi kommer att titta närmare på strängar senare. Men en sträng är också ett värde, därför kan vi använda strängar i villkorsoperatorn.

P.S.

Här är ett program som skriver ut en enkel tabell över tecknen och några av deras tillhörigheter, enligt funktionerna `isalnum()`, etc.

```
1 #include <stdio.h>
2 #include <ctype.h>
3
4 main()
5 {
```

```
6     int ch;
7
8     for (ch = 0; ch < 128; ch++){
9         printf("%3d[%c]", ch, ch);
10        printf(isspace(ch) ? " spc" : "    ");
11        printf(ispunct(ch) ? " pct" : "    ");
12        printf(isalnum(ch) ? " aln" : "    ");
13        printf(isprint(ch) ? " prt" : "    ");
14        printf(isgraph(ch) ? " gph" : "    ");
15        printf(iscntrl(ch) ? " ctl" : "    ");
16        printf(isascii(ch) ? " asc" : "    ");
17        printf("\n");
18    }
19 }
```

Jag använde programmet för att ta reda på vad jag skulle skriva i tabellen tidigare. Utdata från programmet är så långt och trist och oredigerat att det finns i en [egen fil](#). D.S.

Nästa lektion

